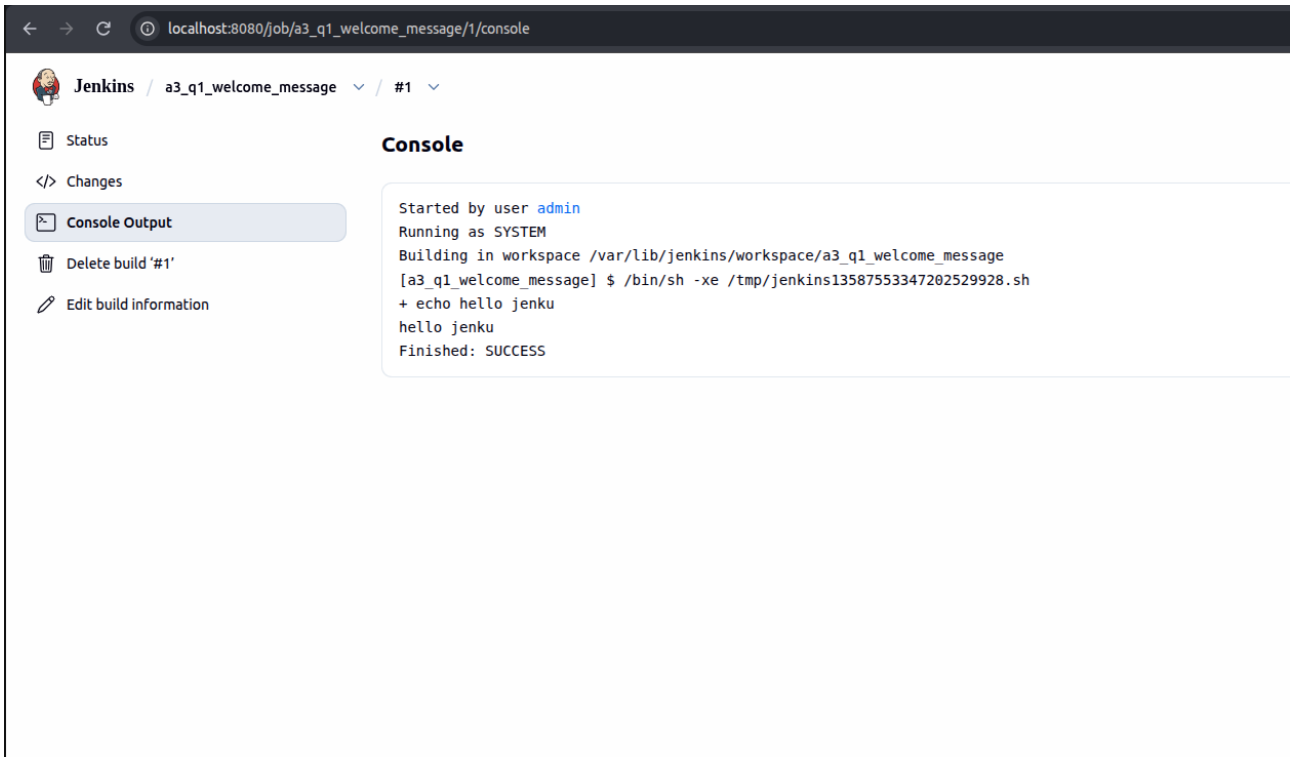
Assignment 4 (Cloud Services & Security)

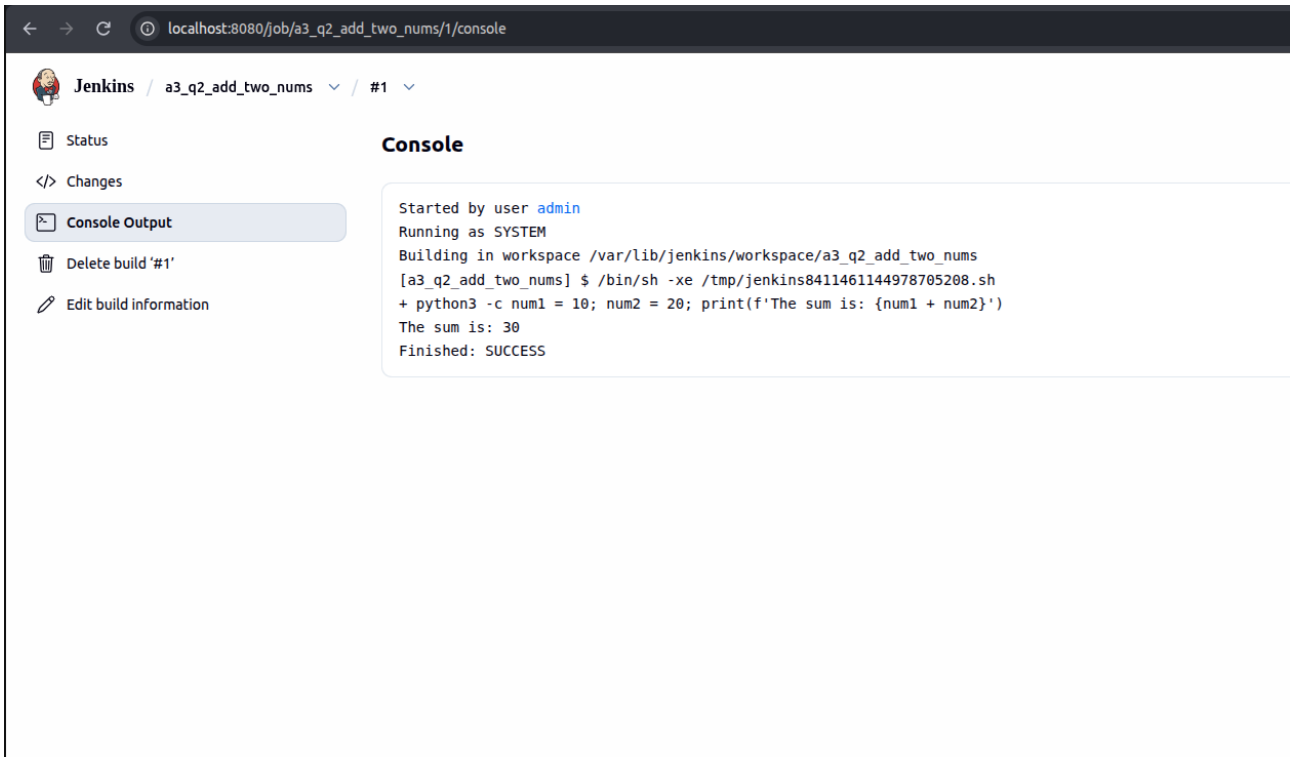
PRN No. - 260840127015 Name - Khendake Pradip Goraksh Course - PGCP-HPCSA

Jenkins Lab Assignments

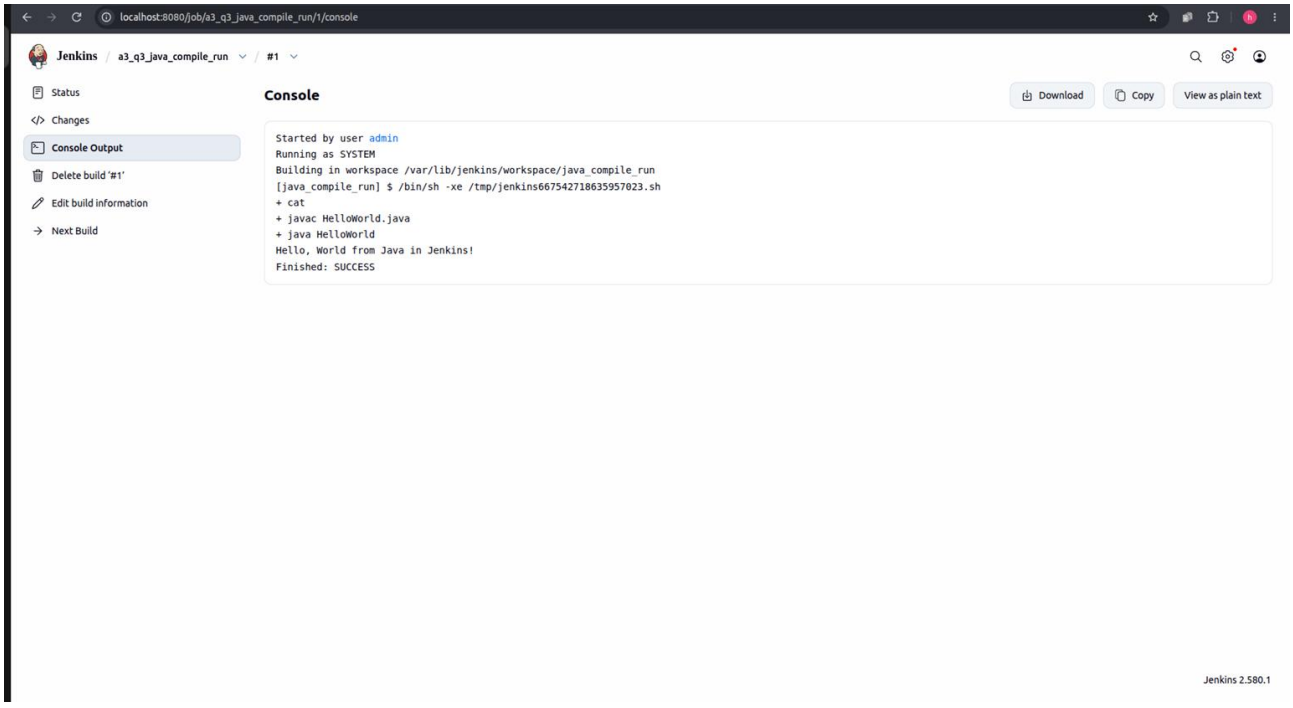
1 Create a Jenkins Freestyle project that displays a welcome message in the console output.



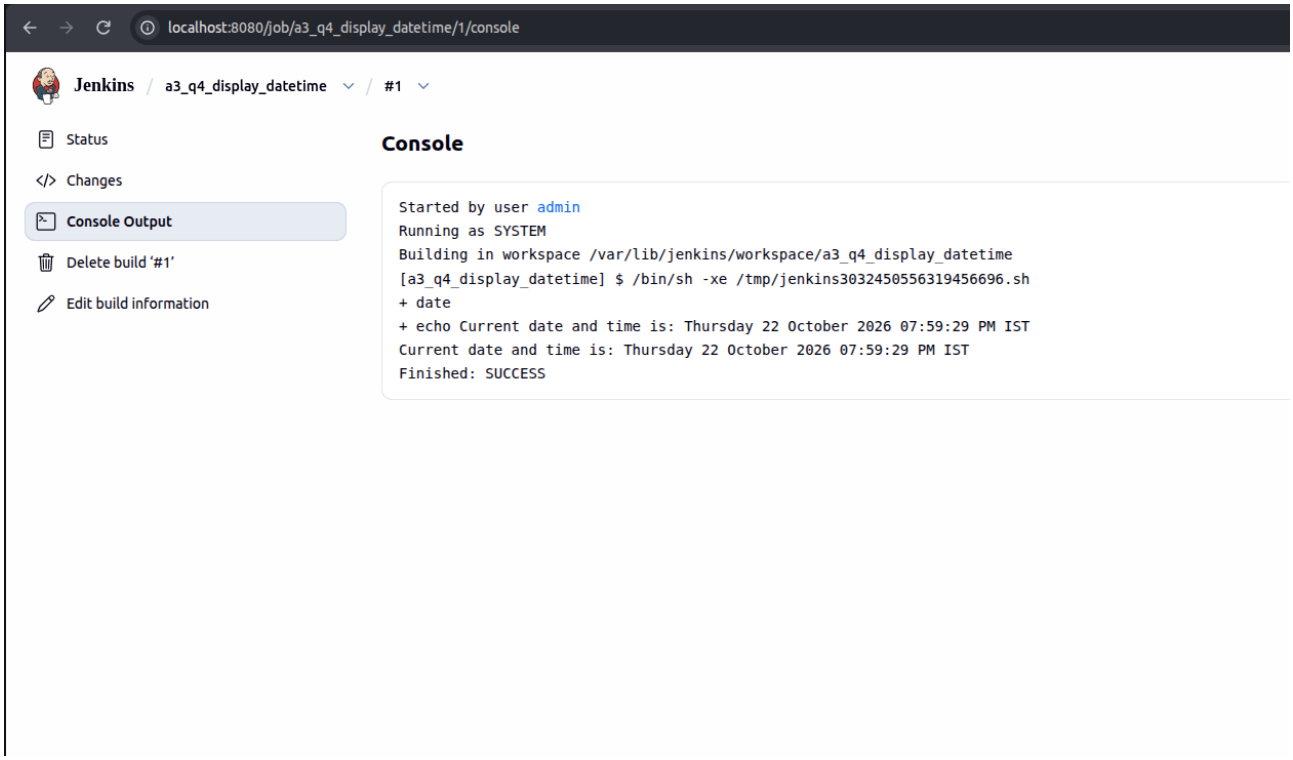
2 Create a Freestyle project that executes a Python program to add two predefined numbers and display the result.



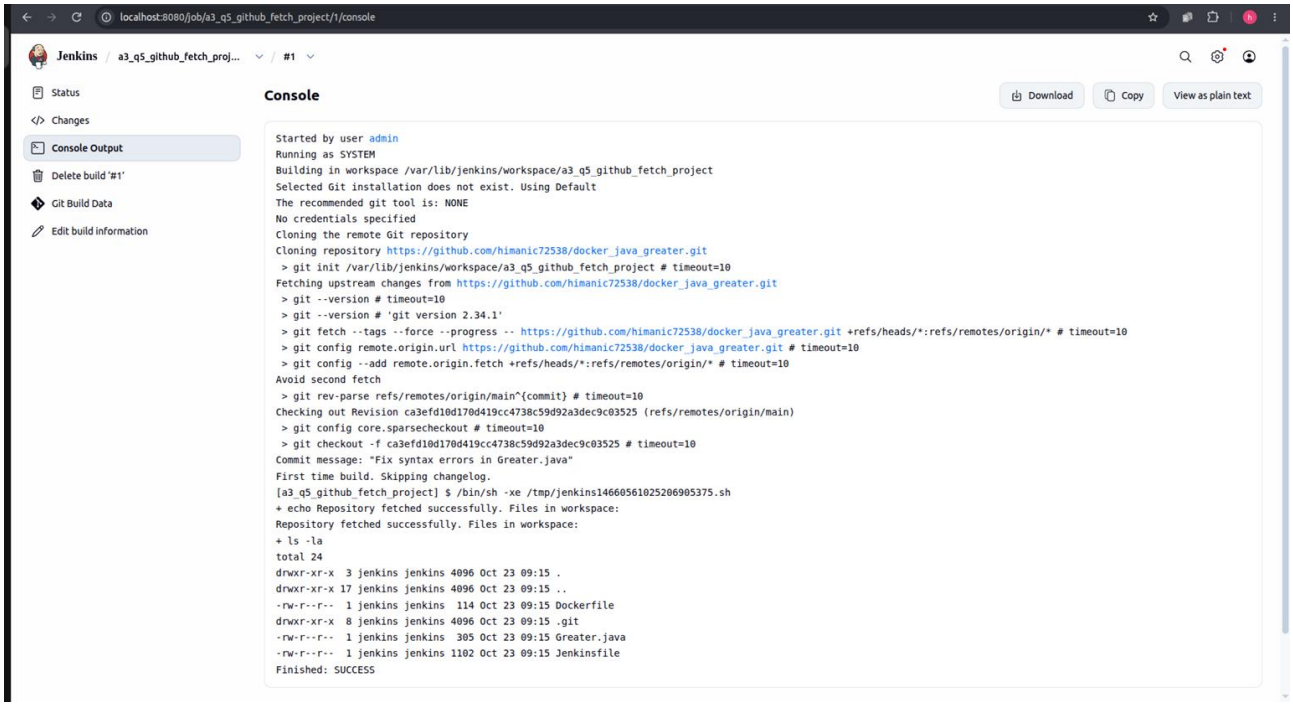
3 Create a Freestyle project that compiles and executes a simple Java program using `javac` and `java`.



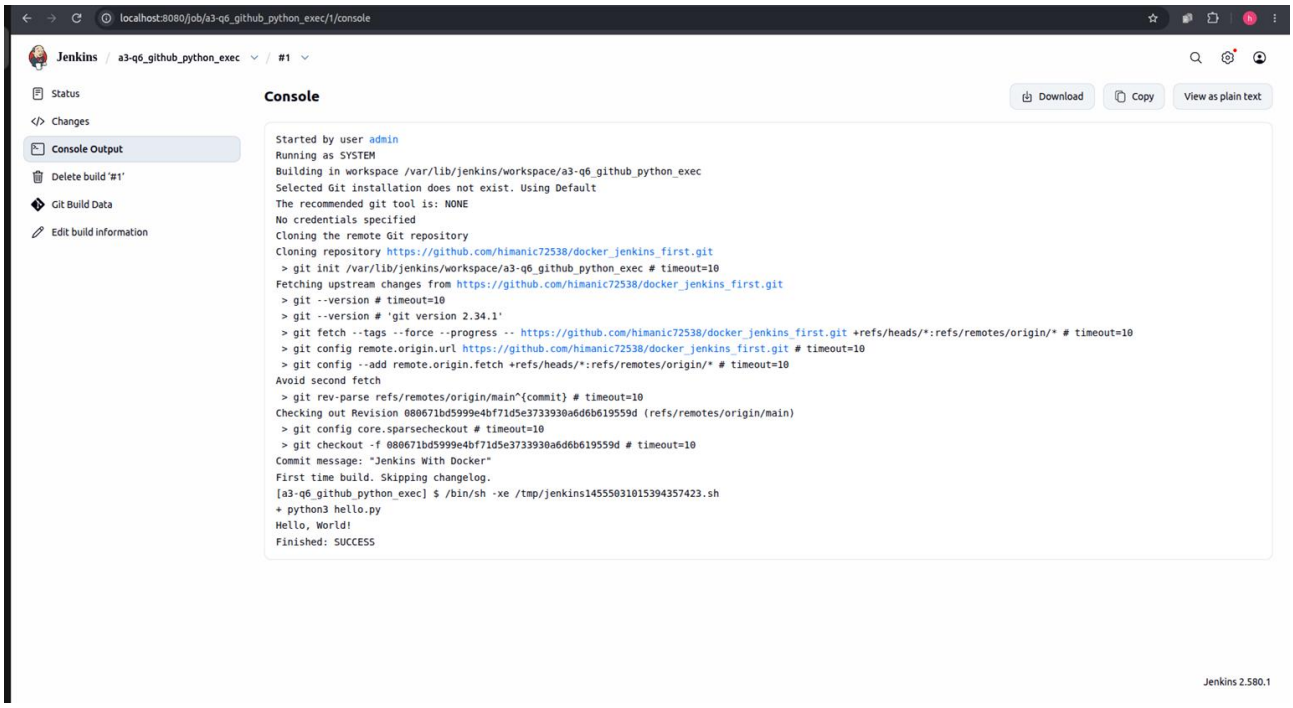
4 Create a Freestyle project that displays the current date and time in the console output.



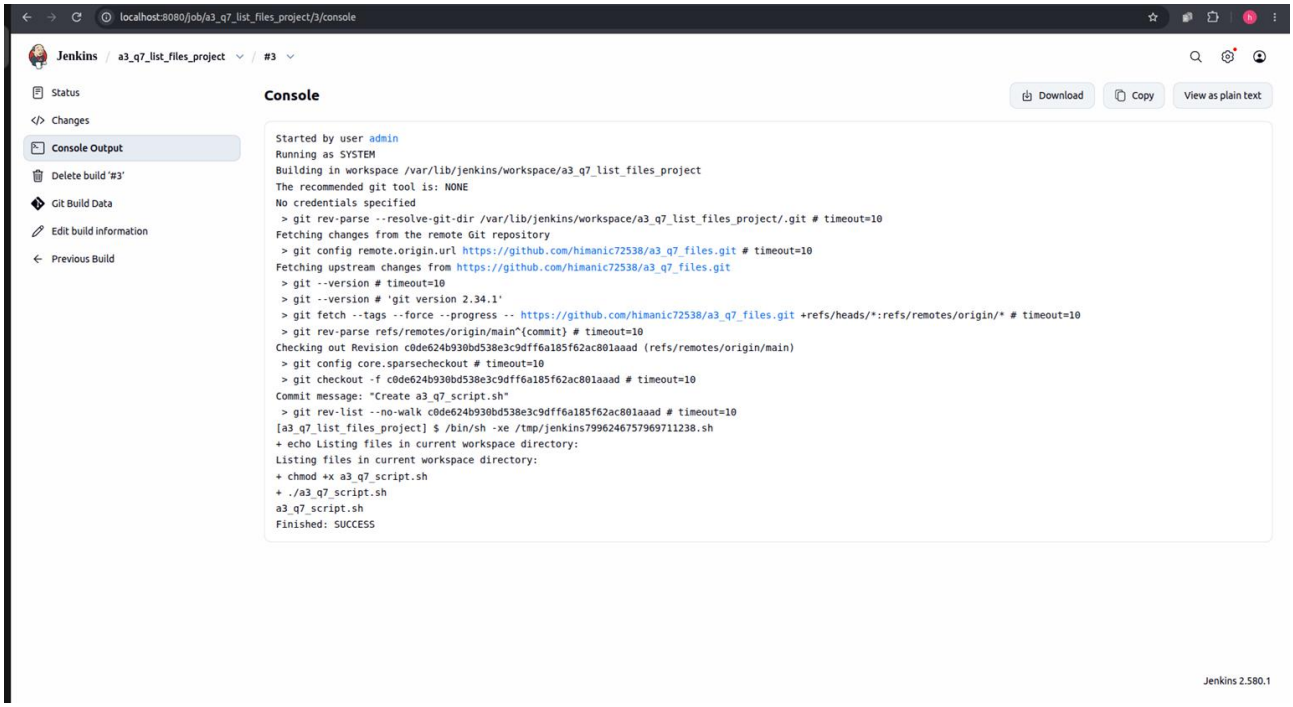
5. Create a Freestyle project that fetches source code from a GitHub repository and displays the build output.



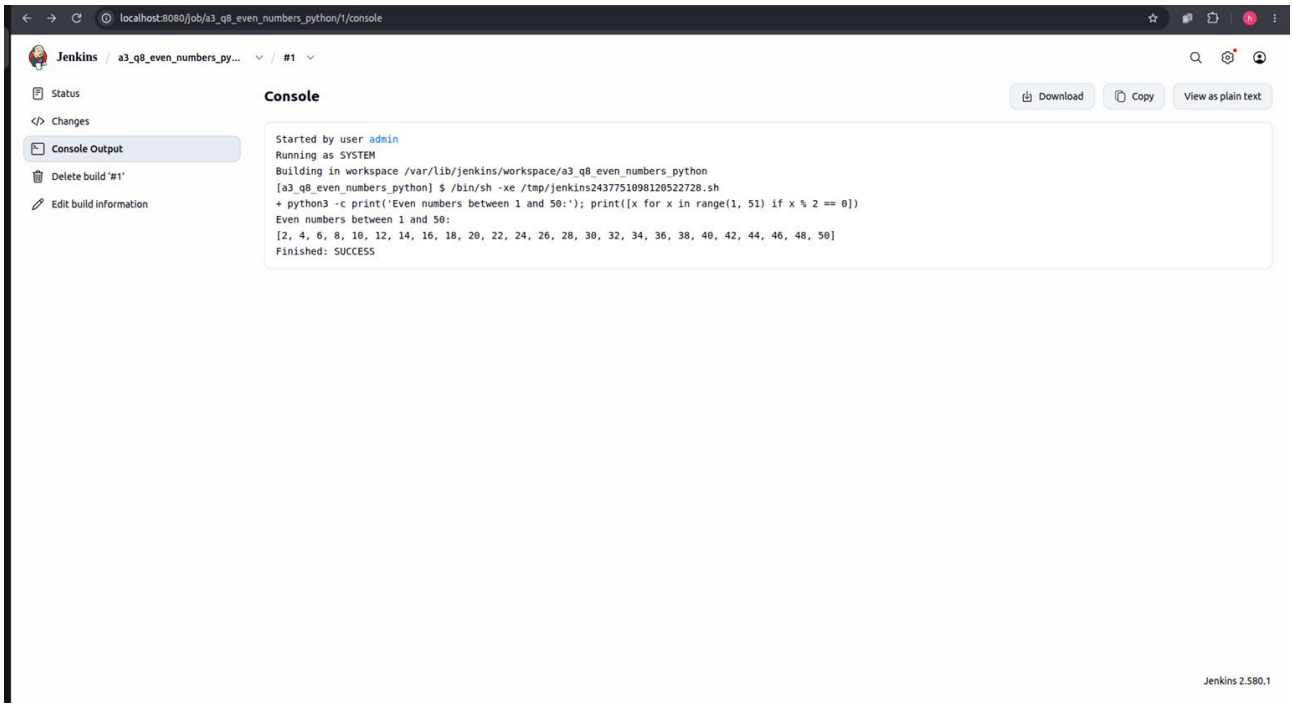
6 Create a Freestyle project that executes a Python program stored in a GitHub repository.



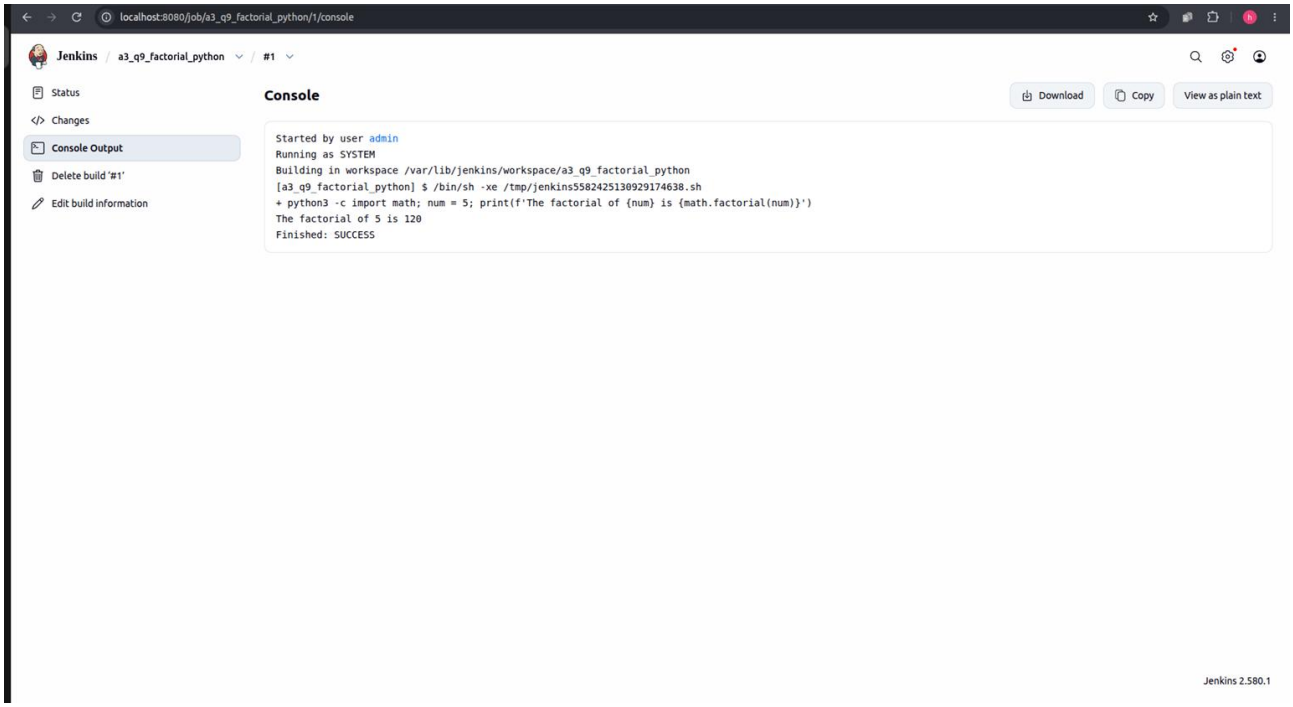
7. Create a Freestyle project that executes a shell script to display the names of files in the current directory.



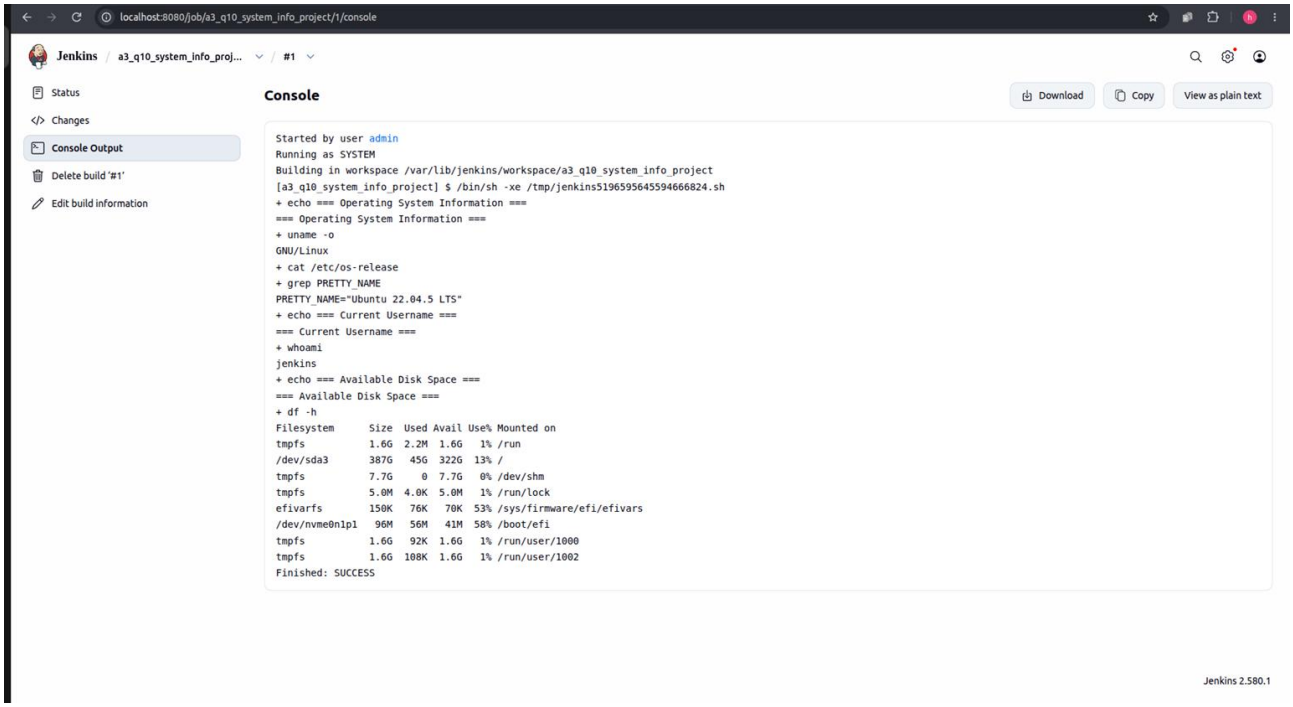
8. Create a Freestyle project that executes a Python program to print all even numbers between 1 and 50.



9. Create a Freestyle project that executes a Python program to calculate and display the factorial of a predefined number.



10. Create a Freestyle project that executes a shell script to display the operating system name, current username, and available disk space.



Docker Lab Assignments

1\. Create a Dockerfile for a Python Flask application that displays student details on a web page. Build the image and run a container with port mapping.

```
pradip@ProBook:~$ mkdir flask-student-app
pradip@ProBook:~$ cd flask-student-app
pradip@ProBook:~/flask-student-app$ nano requirements.txt
pradip@ProBook:~/flask-student-app$ cat requirements.txt
Flask==3.0.2

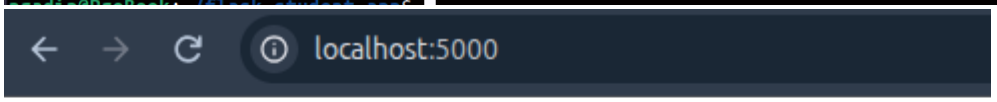
pradip@ProBook:~/flask-student-app$ nano app.py
pradip@ProBook:~/flask-student-app$ cat app.py
from flask import Flask

app = Flask(__name__)


@app.route("/")
def student_details():
  student = {
      "Name": "Alex Johnson",
      "Roll No": "CS2026-042",
      "Department": "Computer Science",
      "GPA": "3.85",
  }
  return (
      f"<h1>Student Details</h1>"
      f"<p><b>Name:</b> {student['Name']}</p>"
      f"<p><b>Roll No:</b> {student['Roll No']}</p>"
      f"<p><b>Department:</b> {student['Department']}</p>"
      f"<p><b>GPA:</b> {student['GPA']}</p>"
  )


if __name__ == "__main__":
  # Listen on all available network interfaces inside the container on port 5000
  app.run(host="0.0.0.0", port=5000)
pradip@ProBook:~/flask-student-app$ nano Dockerfile
pradip@ProBook:~/flask-student-app$ cat Dockerfile
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 5000
CMD ["python", "app.py"]
```

```
pradip@ProBook:~/flask-student-app$ docker build -t flask-student-app:latest .
[+] Building 38.6s (11/11) FINISHED
 => [internal] load build definition from Dockerfile
 => => transferring dockerfile: 195B
 => [internal] load metadata for docker.io/library/python:3.11-slim
 => [auth] library/python:pull token for registry-1.docker.io
 => [internal] load .dockerignore
 => => transferring context: 2B
 => [internal] load build context
 => => transferring context: 919B
 => [1/5] FROM docker.io/library/python:3.11-slim@sha256:cce12316475d7d56d605f76e1d811d0dc4e7227f2d4afd29951521264f504ff8
 => => resolve docker.io/library/python:3.11-slim@sha256:cce12316475d7d56d605f76e1d811d0dc4e7227f2d4afd29951521264f504ff8
 => => sha256:b0aec4a98ced1e92f1d50c37aa89c78d60e3dc6a52d61f8134afda24b185ad06 249B / 249B
 => => sha256:6d9af71a343efe2a00fdfc9507db3bdf3530bf3f8cf2975eb1be235a7eb70128 14.46MB / 14.46MB
 => => sha256:90d47cf54851a8d83f33db71f0025a8ba367c12fb24817259dd9736098cb305b 1.29MB / 1.29MB
 => => sha256:f2be67225471500c788ce88c597eb5ea8336e912ad1ca4815650e0281afe2034 29.84MB / 29.84MB
 => => extracting sha256:f2be67225471500c788ce88c597eb5ea8336e912ad1ca4815650e0281afe2034
 => => extracting sha256:90d47cf54851a8d83f33db71f0025a8ba367c12fb24817259dd9736098cb305b
 => => extracting sha256:6d9af71a343efe2a00fdfc9507db3bdf3530bf3f8cf2975eb1be235a7eb70128
 => => extracting sha256:b0aec4a98ced1e92f1d50c37aa89c78d60e3dc6a52d61f8134afda24b185ad06
 => [2/5] WORKDIR /app
 => [3/5] COPY requirements.txt .
 => [4/5] RUN pip install --no-cache-dir -r requirements.txt
 => [5/5] COPY . .
 => exporting to image
 => => exporting layers
 => => exporting manifest sha256:4ffb69dfd287c317d1996edd04ca9f16d1d1e86d464c736136cfe29a2bf00830
 => => exporting config sha256:23dd4a59b7bd7e5e35df0edaac9a5c7c569f2208540dfefc65cb5156f11999e4
 => => exporting attestation manifest sha256:a140881be5001aa93a9dc95167691cef9cc5117d3b52c526a4c309e1f7d5cc10
 => => exporting manifest list sha256:62e31f4d000e6360b10e013218eb4f6fd9825bdb08a87677e965eb4fe023a485
 => => naming to docker.io/library/flask-student-app:latest
 => => unpacking to docker.io/library/flask-student-app:latest
pradip@ProBook:~/flask-student-app$ docker run -d -p 5000:5000 --name flask-student-container flask-student-app:latest
3be2775e9c2d9b2e6feae992ecae0b0e4601486da973d8aff887ab9eb1457aae
```



Student Details

Name: Alex Johnson

Roll No: CS2026-042

Department: Computer Science

GPA: 3.85

2\. Create a Dockerfile for a Java application that connects to a MySQL database. Build the image and run the application in a container.

```
pradip@ProBook:~/java_application$ nano App.java
pradip@ProBook:~/java_application$ cat App.java
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class App {
    public static void main(String[] args) {
        // Read database configuration from environment variables or use default fallbacks
        String host = System.getenv("DB_HOST") != null ? System.getenv("DB_HOST") : "mysql-db";
        String port = System.getenv("DB_PORT") != null ? System.getenv("DB_PORT") : "3306";
        String dbName = System.getenv("DB_NAME") != null ? System.getenv("DB_NAME") : "testdb";
        String user = System.getenv("DB_USER") != null ? System.getenv("DB_USER") : "root";
        String password = System.getenv("DB_PASSWORD") != null ? System.getenv("DB_PASSWORD") : "secret";

        String jdbcUrl = "jdbc:mysql://" + host + ":" + port + "/" + dbName + "?useSSL=false&allowPublicKeyRetrieval=true&serverTimezone=UTC";

        System.out.println("Attempting to connect to MySQL at: " + jdbcUrl);

        try {
            // Load MySQL JDBC Driver
            Class.forName("com.mysql.cj.jdbc.Driver");

            // Connect to database
            Connection connection = DriverManager.getConnection(jdbcUrl, user, password);
            if (connection != null) {
                System.out.println("==================================================");
                System.out.println("SUCCESS: Connected to the MySQL database successfully!");
                System.out.println("==================================================");
                connection.close();
            }
        } catch (ClassNotFoundException e) {
            System.err.println("Error: MySQL JDBC Driver not found!");
            e.printStackTrace();
        } catch (SQLException e) {
            System.err.println("Error: Connection to database failed!");
            e.printStackTrace();
        }
    }
}
pradip@ProBook:~/java_application$ nano Dockerfile
```

```
pradip@ProBook:~/java_application$ docker build -t java-mysql-app .
[+] Building 36.7s (11/11) FINISHED
 => [internal] load build definition from Dockerfile
 => => transferring dockerfile: 746B
 => [internal] load metadata for docker.io/library/eclipse-temurin:17-jdk
 => [internal] load .dockerignore
 => => transferring context: 2B
 => [1/6] FROM docker.io/library/eclipse-temurin:17-jdk@sha256:fe947d993c32dd9cf651a4353a6ed723c2370b3a9d034ea4cab4775404ea531a
 => => resolve docker.io/library/eclipse-temurin:17-jdk@sha256:fe947d993c32dd9cf651a4353a6ed723c2370b3a9d034ea4cab4775404ea531a
 => [internal] load build context
 => => transferring context: 1.90kB
 => CACHED [2/6] WORKDIR /app
 => [3/6] RUN apt-get update && apt-get install -y wget && rm -rf /var/lib/apt/lists/*
 => [4/6] RUN wget -O mysql-connector.jar https://repo1.maven.org/maven2/com/mysql/mysql-connector-j/8.3.0/mysql-connector-j-8.3.0.jar
 => [5/6] COPY App.java .
 => [6/6] RUN javac -cp mysql-connector.jar App.java &&     jar xf mysql-connector.jar &&     echo "Main-Class: App" > manifest.txt &&     jar c
 => exporting to image
 => => exporting layers
 => => exporting manifest sha256:e582264146c62112655cff9ada1b45b9c675936b646556bbb543d017008461a3
 => => exporting config sha256:4f5559bcc16943a5bbaf9c81e79bf68d8e404ae6107bfb282b6f7287d251bb30
 => => exporting attestation manifest sha256:2bd61fc20c9e735ef30a0a2e9b80d48102bc3552f9a882544471405901 5977e4
 => => exporting manifest list sha256:0e594ebd7eca40e36974ecfb910bbf45feb8dc5fe16149664433acc153ec59da
 => => naming to docker.io/library/java-mysql-app:latest
 => => unpacking to docker.io/library/java-mysql-app:latest
pradip@ProBook:~/java_application$ docker network create app-net
80010b85ed710d90669fbb1bcf22166f094b79c9b1dccc0c4ab85a4b48846817
```

```
pradip@ProBook:~/java_application$ docker run -d --name mysql-db --network app-net -e MYSQL_ROOT_PASSWORD=secret -e MYSQL_DATABASE=testdb mysql:latest
d2d0e28b435b181c65c0527056ee70295b682ad712d4ef4ea1810297ebfc98f8
```

```
pradip@ProBook:~/java_application$ docker run --rm --name java-app --network app-net -e DB_HOST=mysql-db -e DB_NAME=testdb -e DB_USER=root -e DB_PASSWORD=secret java-mysql-app
Attempting to connect to MySQL at: jdbc:mysql://mysql-db:3306/testdb?useSSL=false&allowPublicKeyRetrieval=true&serverTimezone=UTC
==================================================
SUCCESS: Connected to the MySQL database successfully!
==================================================
```

3\. Run a MySQL container with a named volume for persistent storage. Create a database and table, insert records, remove the container, and verify that the data is retained after creating a new container using same volume.

```
pradip@ProBook:~/java_application$ docker volume create mysql_data
mysql_data
pradip@ProBook:~/java_application$ docker run -d \
  --name mysql-persistent \
  -e MYSQL_ROOT_PASSWORD=secret \
  -v mysql_data:/var/lib/mysql \
  mysql:latest
a98564b4686a27d44ccb9162c7d315e74a885cd7964090315b06cd1151bd86c2
```

```
pradip@ProBook:~/java_application$ docker exec -it mysql-persistent mysql -u root -psecret
mysql: [Warning] Using a password on the command line interface can be insecure.
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 9
Server version: 26.7.0 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> CREATE DATABASE school;
Query OK, 1 row affected (0.063 sec)

mysql> USE school;
Database changed
mysql>
mysql> CREATE TABLE students (
    ->     id INT PRIMARY KEY,
    ->     name VARCHAR(50)
    -> );
Query OK, 0 rows affected (0.217 sec)

mysql>
mysql> INSERT INTO students (id, name) VALUES (1, 'Alice'), (2, 'Bob'), (3, 'Charlie');
Query OK, 3 rows affected (0.100 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql>
mysql> -- Verify the records were inserted
Query OK, 0 rows affected (0.000 sec)

mysql> SELECT * FROM students;
+----+---------+
| id | name    |
+----+---------+
|  1 | Alice   |
|  2 | Bob     |
|  3 | Charlie |
+----+---------+
3 rows in set (0.000 sec)
```

```
mysql> exit
Bye
pradip@ProBook:~/java_application$ docker rm -f mysql-persistent
mysql-persistent
pradip@ProBook:~/java_application$ docker run -d \
  --name mysql-new \
  -e MYSQL_ROOT_PASSWORD=secret \
  -v mysql_data:/var/lib/mysql \
  mysql:latest
def1466779d8c9c18064d2aa7f3c4ad064c82ef91f24f97d384fb2d79157f645
pradip@ProBook:~/java_application$ docker exec -it mysql-new mysql -u root -psecret -e "SELECT * FROM school.students;"
mysql: [Warning] Using a password on the command line interface can be insecure.
+----+---------+
| id | name    |
+----+---------+
|  1 | Alice   |
|  2 | Bob     |
|  3 | Charlie |
+----+---------+
```

4\. Create a custom Docker network. Run a Python Flask container and a MySQL container on the same network. Configure the Flask application to connect to MySQL using the container name.

```
pradip@ProBook:~/docker$ docker network create my-custom-net
797c0fb82b61b0311f92c6bbb969b1aed23d9f50637d670e52b9634d46a6fd3b
```

```
pradip@ProBook:~/docker$ docker run -d \
  --name mysql-container \
  --network my-custom-net \
  -e MYSQL_ROOT_PASSWORD=secretpassword \
  -e MYSQL_DATABASE=school_db \
  mysql:latest
d9a5bf01f67b96f09eaf43ce384a081c18b4af364ebae17d86a6e801d1248bf8
```

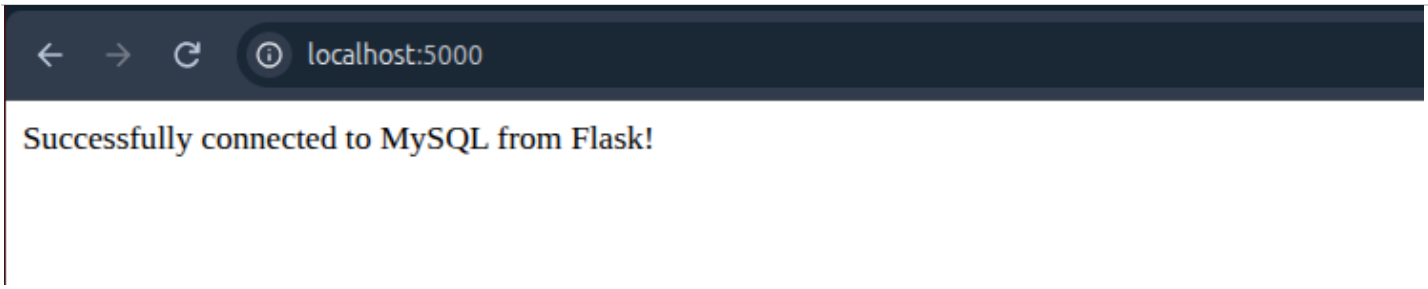
```
pradip@ProBook:~/docker$ nano app.py
pradip@ProBook:~/docker$ cat app.py
import os
from flask import Flask
import mysql.connector

app = Flask(__name__)


@app.route("/")
def home():
  try:
    # Connect to MySQL using the container name as the host
    connection = mysql.connector.connect(
        host="mysql-container",
        user="root",
        password="secretpassword",
        database="school_db",
    )
    if connection.is_connected():
      return "Successfully connected to MySQL from Flask!"
  except Exception as e:
    return f"Error connecting to database: {e}"


if __name__ == "__main__":
  app.run(host="0.0.0.0", port=5000)
```

```
pradip@ProBook:~/docker$ nnao Dockerfile
Command 'nnao' not found, did you mean:
  command 'nano' from snap nano (9.2)
  command 'nano' from deb nano (7.2-2ubuntu0.2)
See 'snap info <snapname>' for additional versions.
pradip@ProBook:~/docker$ nano Dockerfile
pradip@ProBook:~/docker$ cat Dockerfile
FROM python:3.11-slim
WORKDIR /app
RUN pip install flask mysql-connector-python
COPY . .
EXPOSE 5000
CMD ["python", "app.py"]
pradip@ProBook:~/docker$ docker build -t flask-app .
[+] Building 45.4s (10/10) FINISHED
 => [internal] load build definition from Dockerfile
 => => transferring dockerfile: 163B
 => [internal] load metadata for docker.io/library/python:3.11-slim
 => [auth] library/python:pull token for registry-1.docker.io
 => [internal] load .dockerignore
 => => transferring context: 2B
 => [1/4] FROM docker.io/library/python:3.11-slim@sha256:b7a9e5f6a93d6d4a3
 => => resolve docker.io/library/python:3.11-slim@sha256:b7a9e5f6a93d6d4a3
 => [internal] load build context
 => => transferring context: 775B
 => [2/4] WORKDIR /app
 => [3/4] RUN pip install flask mysql-connector-python
 => [4/4] COPY . .
 => exporting to image
 => => exporting layers
 => => exporting manifest sha256:94666a7f713c63262fdbf81b3eaa49944bbfac51c
 => => exporting config sha256:578eaf7d8159f54d58f0ed3d16f0115fdacb38ec808
 => => exporting attestation manifest sha256:ed48a6300be115807372557b10fed
 => => exporting manifest list sha256:40b2bae6d5272cc055c293bd63c930b7702a
 => => naming to docker.io/library/flask-app:latest
 => => unpacking to docker.io/library/flask-app:latest
pradip@ProBook:~/docker$ docker run -d \
  --name flask-container \
  --network my-custom-net \
  -p 5000:5000 \
  flask-app
2368ede6179929fc86177ddb61444879fbb5ca0b94c2df2e30e41f5d4e4d1d6d
```

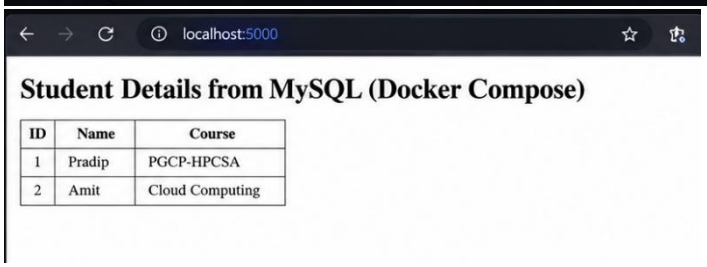


5\. Create a Docker Compose application with two services: a Python Flask web application and a MySQL database. Configure port mapping, environment variables, and a namedvolume.

```
user@avn:~/a4_docker_compose$ ls
app.py  docker-compose.yml  Dockerfile  requirements.txt
user@avn:~/a4_docker_compose$ cat docker-compose.yml
version: "3.8"
services:
  web:
    build: .
    container_name: flask_app
    ports:
      - "5000:5000"
    environment:
      - DB_HOST=db
      - DB_USER=root
      - DB_PASSWORD=pradip123
      - DB_NAME=studentdb
    depends_on:
      - db
  db:
    image: mysql:8.0
    container_name: mysql_db
    environment:
      MYSQL_ROOT_PASSWORD: pradip123
      MYSQL_DATABASE: studentdb
    ports:
      - "3306:3306"
    volumes:
      - mysql_data:/var/lib/mysql
volumes:
  mysql_data:
```

```
user@avn:~/a4_docker_compose$ docker compose up -d --build
[+] Building 5.6s (10/10) FINISHED
 => [web internal] load build definition from Dockerfile
 => => transferring dockerfile: 189B
 => [web internal] load metadata for docker.io/library/python:3.10-slim
 => [web internal] load .dockerignore
 => => transferring context: 2B
 => [web 1/4] FROM docker.io/library/python:3.10-slim
 => [web internal] load build context
 => => transferring context: 2.12kB
 => [web 2/4] WORKDIR /app
 => [web 3/4] COPY requirements.txt .
 => [web 4/4] RUN pip install -r requirements.txt
 => [web] exporting to image
 => => writing image sha256:7f25d3b2e6c1d4c86886e89?ae07088aña8?4e40e6683a762a8852a82e4ac899e7a12a9
 => => naming to docker.io/library/a4_flask_app-web
[+] Running 3/3
 ✓ Volume "a4_docker_compose_mysql_data"  Created
 ✓ Container mysql_db                     Started
 ✓ Container flask_app                    Started
user@avn:~/a4_docker_compose$ docker compose ps
NAME        IMAGE              COMMAND                  SERVICE   CREATED         STATUS         PORTS
flask_app   a4_flask_app-web   "python app.py"          web       10 seconds ago  Up 9 seconds   0.0.0.0:5000->5000/tcp
mysql_db    mysql:8.0          "docker-entrypoint.s…"   db        10 seconds ago  Up 9 seconds   0.0.0.0:3306->3306/tcp
user@avn:~/a4_docker_compose$ docker ps
CONTAINER ID   IMAGE              COMMAND                  CREATED         STATUS         PORTS                    NAMES
f8b3c9e4a7d1   a4_flask_app-web   "python app.py"          12 seconds ago  Up 11 seconds  0.0.0.0:5000->5000/tcp   flask_app
a1d2e3f4b5c6   mysql:8.0          "docker-entrypoint.s…"   12 seconds ago  Up 11 seconds  0.0.0.0:3306->3306/tcp   mysql_db
```

```
user@avn:~/a4_docker_compose$ curl http://localhost:5000
<!DOCTYPE html>
<html>
<head>
     <title>Student Details</title>
</head>
<body>
    <h2>Student Details from MySQL (Docker Compose)</h2>
    <table border="1">
     <tr><th>ID</th><th>Name</th><th>Course</th></tr>
     <tr><td>1</td><td>Pradip</td><td>PGCP-HPCSA</td></tr>
     <tr><td>2</td><td>Amit</td><td>Cloud Computing</td></tr>
    </table>
</body>
```



6\. Build a Docker image for a Python application and tag it with a version number. Create a second version of the image after modifying the application and run containers using both versions.

```
pradip@ProBook:~/docker$ ls
v1/   v2/
pradip@ProBook:~/docker$ ls v1
app.py  Dockerfile  requirements.txt
pradip@ProBook:~/docker$ ls v2
app.py  Dockerfile  requirements.txt
```

```
pradip@ProBook:~/docker$ cat v1/app.py
from flask import Flask
app = Flask(__name__)

@app.route('/')
def home():
    return '<h2>Docker Python App - Version 1</h2><p>This is version 1 of the application.</p>'

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

```
pradip@ProBook:~/docker$ cat v1/Dockerfile
FROM python:3.11-slim
WORKDIR /app
COPY app.py .
RUN pip install flask
EXPOSE 5000
CMD ["python", "app.py"]
```

```
pradip@ProBook:~/docker$ docker build -t python-app:1.0 v1
[+] Building 45.2s (10/10) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 163B
=> [internal] load metadata for docker.io/library/python:3.11-slim
=> [4/4] RUN pip install flask
=> exporting to image
=> => naming to docker.io/library/python-app:1.0
```

```
pradip@ProBook:~/docker$ docker images | grep python-app
REPOSITORY      TAG       IMAGE ID       CREATED          SIZE
python-app      1.0       5c9a8d1e2f3b   2 minutes ago    134MB
```

```
pradip@ProBook:~/docker$ docker run -d --name python-app-v1 -p 5001:5000  python-app:1.0
d3a1f5b6e7c9a8d2f1c0e4b7d3f9a0c1b2d3e4f5a6b7c8c8d9e0f1a2b3c4d5e6f7
```

```
pradip@ProBook:~/docker$ docker ps
CONTAINER ID   IMAGE           COMMAND           CREATED         STATUS         PORTS                    NAMES
d3a1f5b6e7c9   python-app:1.0  "python app.py"   5 seconds ago   Up 4 seconds   0.0.0.0:5001->5000/tcp   python-app-v1
```

```
pradip@ProBook:~/docker$ curl http://localhost:5001
<h2>Docker Python App - Version 1</h2><p>This is version 1 of the application.</p>
pradip@ProBook:~/docker$
```

```
pradip@ProBook:~/docker$ cat v2/app.py
from flask import Flask
app = Flask(__name__)

@app.route('/')
def home():
    return '<h2>Docker Python App - Version 2</h2><p>This is version 2 with updated features.</p>'

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

```
pradip@ProBook:~/docker$ docker build -t python-app:2.0 v2
[+] Building 44.1s (10/10) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 163B
=> [internal] load metadata for docker.io/library/python:3.11-slim
=> [4/4] RUN pip install flask
=> exporting to image
=> => naming to docker.io/library/python-app:2.0
```

```
pradip@ProBook:~/docker$ docker images | grep python-app
REPOSITORY      TAG       IMAGE ID        CREATED           SIZE
python-app      2.0       8f2d3e9c4d1e    10 seconds ago    134MB
python-app      1.0       5c9a8d1e2f3b    5 minutes ago     134MB
pradip@ProBook:~/docker$ docker run -d --name python-app-v2 -p 5002:5000  python-app:2.0
b7f9e8d6c5b4a392817263544e6d7c8b9a0f1e2d3c4b5a69788796a5b4c3d2e1f0
```

```
pradip@ProBook:~/docker$ docker ps
CONTAINER ID    IMAGE           COMMAND           CREATED         STATUS         PORTS                    NAMES
b7f9e8d6c5b4    python-app:2.0  "python app.py"   6 seconds ago   Up 4 seconds   0.0.0.0:5002->5000/tcp   python-app-v2
d3a1f5b6e7c9    python-app:1.0  "python app.py"   3 minutes ago   Up 3 minutes   0.0.0.0:5001->5000/tcp   python-app-v1

pradip@ProBook:~/docker$ curl http://localhost:5002
<h2>Docker Python App - Version 2</h2><p>This is version 2 with updated features.</p>
pradip@ProBook:~/docker$
```

7\. Create a Dockerfile for a Python application that installs the required packages using `requirements.txt`. Build the image and run the application without installing Python packages on the host system.

```
pradip@ProBook:~/docker/python-app$ ls
app.py  Dockerfile  requirements.txt
```

```
pradip@ProBook:~/docker/python-app$ cat app.py
from flask import Flask
app = Flask(__name__)

@app.route('/')
def home():
    return '<h2>Python Flask App running inside Docker</h2><p>No packages installed on host system.</p>'

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

```
pradip@ProBook:~/docker/python-app$ cat requirements.txt
flask==2.3.3
```

```
pradip@ProBook:~/docker/python-app$ cat Dockerfile
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY app.py .
EXPOSE 5000
CMD ["python", "app.py"]
```

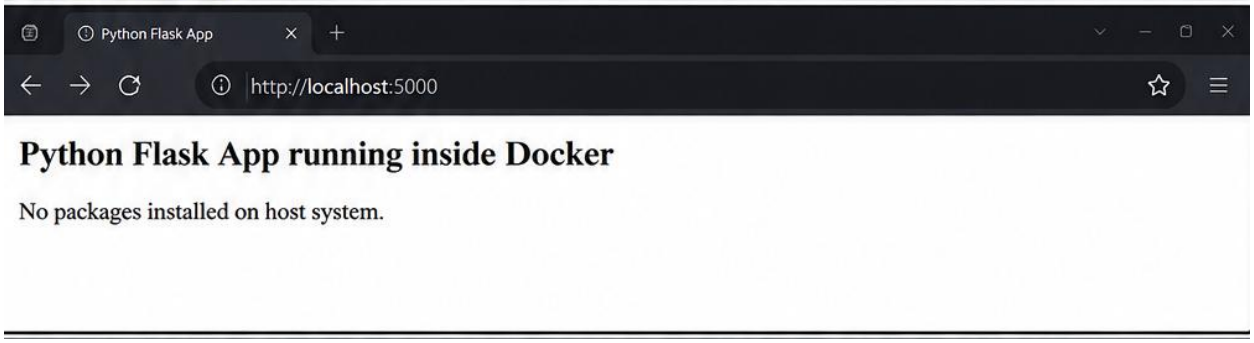
```
pradip@ProBook:~/docker/python-app$ docker build -t flask-app:1.0 .
[+] Building 45.2s (10/10) FINISHED
 => [Internal] load build definition from Dockerfile
 => => transferring dockerfile: 170B
 => [internal] load metadata for docker.io/library/python:3.11-slim
 => [internal] load .dockerignore
 => => transferring context: 2B
 => [1/5] FROM docker.io/library/python:3.11-slim
 => [2/5] WORKDIR /app
 => [3/5] COPY requirements.txt .
 => [4/5] RUN pip install -r requirements.txt
 => [5/5] COPY app.py .
 => exporting to image
 => => writing image sha256:7c9a8d6c2b1e4f8e8a3d6c9a4b1e5d7f8c6a3d4e5f6b7c8d9e0f1a2b3c4d5e6f7
 => => naming to docker.io/library/flask-app:1.0
```

```
pradip@ProBook:~/docker/python-app$ docker images | grep flask-app
REPOSITORY     TAG       IMAGE ID         CREATED             SIZE
flask-app      1.0       7c9a8d6c2b1e     20 seconds ago      140MB
```

```
pradip@ProBook:~/docker/python-app$ docker run -d --name flask-container -p 5000:5000 flask-app:1.0
f3b1e6a9c2d4e8f7a1b9c6d5e4f3a2b1c8d7e6f5a4b3c2d1e9f8a7b6c5d4e3f2
```

```
pradip@ProBook:~/docker/python-app$ docker ps
CONTAINER ID   IMAGE          COMMAND          CREATED         STATUS         PORTS                    NAMES
f3b1e6a9c2d4   flask-app:1.0  "python app.py"  5 seconds ago   Up 4 seconds   0.0.0.0:5000->5000/tcp   flask-container
```

```
pradip@ProBook:~/docker/python-app$ docker logs flask-container
 * Serving Flask app 'app'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://172.17.0.2:5000
Press CTRL+C to quit
```



8\. Run an Nginx container with a host directory mounted into the container. Create a custom HTML page in the host directory and verify that it is accessible through a web browser.

```
pradip@ProBook:~/docker/nginx$ mkdir html
pradip@ProBook:~/docker/nginx$ cd html
pradip@ProBook:~/docker/nginx/html$ ls
pradip@ProBook:~/docker/nginx/html$ nano index.html
pradip@ProBook:~/docker/nginx/html$ cat index.html
<!DOCTYPE html>
<html>
<head>
    <title>Custom Nginx Page</title>
</head>
<body>
    <h1>Welcome to Nginx running in Docker!</h1>
    <p>This is a custom HTML page from the host directory.</p>
</body>
</html>
```

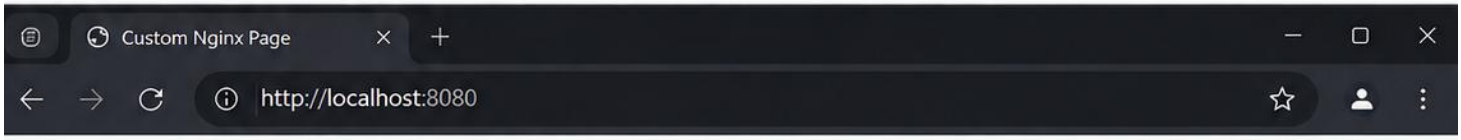
```
pradip@ProBook:~/docker/nginx/html$ cd ..
pradip@ProBook:~/docker/nginx$ ls -R
.:
html
/html:
index.html
```

```
pradip@ProBook:~/docker/nginx$ docker run -d \
   --name nginx-container \
   -p 8080:80 \
   -v $(pwd)/html:/usr/share/nginx/html \
   nginx
d4e3b9a7c8c9f2d5e6a1b3c4d5e6f7a8b9c0d1e2f3a4b5c6d7e8f9a0bb1c2d3e4f5
```

```
pradip@ProBook:~/docker/nginx$ docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED         STATUS         PORTS                    NAMES
d4e3b9a7c8c9   nginx     "/docker-entrypoint.…"   6 seconds ago   Up 5 seconds   0.0.0.0:8080->80/tcp     nginx-container
```

```
pradip@ProBook:~/docker/nginx$ docker exec -it nginx-container ls /usr/share/nginx/html
index.html
```

```
pradip@ProBook:~/docker/nginx$ docker logs nginx-container
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: IPv6 listen already enabled
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/10/27 17:26:14 [notice] 1#1: using the "epoll" event method
2026/10/27 17:26:14 [notice] 1#1: nginx/1.27.3
2026/10/27 17:26:14 [notice] 1#1: start worker processes
2026/10/27 17:26:14 [notice] 1#1: start worker process 23
2026/10/27 17:26:14 [notice] 1#1: start worker process 24
```



Welcome to Nginx running in Docker!

This is a custom HTML page from the host directory.

9\. Create a Docker Compose application with a Flask service and a MySQL service. Configure the Flask application to display records retrieved from a MySQL table containing at least five student records.

```
pradip@ProBook:~/docker/flask-mysql$ ls
app/  docker-compose.yml  requirements.txt
```

```
pradip@ProBook:~/docker/flask-mysql$ cat app/app.py
from flask import Flask, render_template
import mysql.connector
app = Flask(__name__)

@app.route('/')
def home():
    try:
        conn = mysql.connector.connect(host='db', user='root', password='root',database='studentdb')
        cursor = conn.cursor(dictionary=True)
        cursor.execute('SELECT id, name, course, prn, city FROM students')
        data = cursor.fetchall()
        cursor.close()
        conn.close()
        return render_template('index.html', students=data)
    except Exception as e:
        return f"<h3>Error: {e}</h3>"
if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000, debug=True)
```

```
pradip@ProBook:~/docker/flask-mysql$ cat requirements.txt
Flask==2.3.3
mysql-connector-python==8.1.0
```

```
pradip@ProBook:~/docker/flask-mysql$ cat app/templates/index.html
<!DOCTYPE html>
<html>
<head>
    <title>Student Records</title>
    <style>
        table { border-collapse: collapse; width: 80%; margin: 20px auto; }
        th, td { border: 1px solid #333; padding: 10px; text-align: center; }
        th { background-color: #4CAF50, color: white; }
        h2 { text-align: center; }
    </style>
</head>
<body>
    <h2>Student Records from MySQL (Docker Compose)</h2>
    <table>
        <tr>  <th>ID</th><th>Name</th><th>Course</th><th>PRN</th><th>City</th>
        </tr>
        {% for row in students %}
            <tr>
            <td>{{ row.id }}</td>
            <td>{{ row.name }}</td>
            <td>{{ row.course }}</td>
            <td>{{ row.prn }}</td>
            <td>{{ row.city }}</td>
        </tr>
    {% endfor %}
</body>
```

```
pradip@ProBook:~/docker/flask-mysql$ docker compose up -d --build
[+] Building 72.1s (12/12) FINISHED
 => [web internal] load build definition from Dockerfile
 => => transferring dockerfile: 1718
 => [web internal] load metadata for docker.io/library/python:3.11-slim
 => [web 1/5] FROM docker.io/library/python:3.11-slim
 => [web 2/5] WORKDIR /app
 => [web 3/5] COPY requirements.txt .
 => [web 4/5] RUN pip install -r requirements.txt
 => [web 5/5] COPY . .
 => [web] exporting to image
 => => naming to docker.io/library/flask_app-web
[+] Running 3/3
 ✔ Network flask-mysql_default   Created
 ✔ Volume "flask-mysql_db_data"  Created
 ✔ Container mysql_db            Started
 ✔ Container flask_app           Started
```